PROJECT SYNOPSIS

SECUREMAIL: A SECURE CLOUD-BASED EMAIL MANAGEMENT SYSTEM WITH WIRE-LEVEL ENCRYPTION AND ANTI-SPOOFING

Submitted in partial fulfillment of the requirements for the award of the degree of
MASTER OF COMPUTER APPLICATIONS

To
Guru Gobind Singh Indraprastha University, Delhi

Submitted by:

Rohan Kumar Rawat

MCA 3rd Semester • Batch (2025–2027)

Roll No: 02414004425



JAGAN INSTITUTE OF MANAGEMENT STUDIES
3, Institutional Area, Sector-5, Rohini, Delhi-110085
Accredited with NAAC A++ Grade & NBA Accreditation

Batch (2025-2027)

PROJECT PROPOSAL & REGISTRATION DETAILS

Course Code & Title	**MCA-269: Minor Project - III**
Title of Project	**SecureMail: A Secure Cloud-Based Email Management System with Wire-Level Encryption and Anti-Spoofing**
Name of Student	**Rohan Kumar Rawat**
University Roll No.	**02414004425**
Program & Semester	Master of Computer Applications (MCA) – 3rd Semester
Academic Batch	2025 – 2027
Institutional Campus	Jagan Institute of Management Studies (JIMS), Sector-5, Rohini, Delhi
Affiliating University	Guru Gobind Singh Indraprastha University (GGSIPU), Delhi

Declaration by the Candidate

I hereby declare that this project synopsis entitled **"SecureMail: A Secure Cloud-Based Email Management System with Wire-Level Encryption and Anti-Spoofing"** is an authentic, independent record of proposed academic work carried out by me. The embodied work has not formed the basis for the award of any other degree, diploma, or certificate.

Candidate: Rohan Kumar Rawat
Roll Number: 02414004425
Program: Master of Computer Applications (MCA)

1. Title of the Project

> **SecureMail: A Secure Cloud-Based Email Management System with Wire-Level Encryption and Anti-Spoofing**

Domain: Cloud Computing, Cryptographic Network Security, Distributed Systems.
Classification: Sovereign Institutional Mail Platform with Hardware-Accelerated Wire Encryption.

2. Problems with the Existing System

Under conventional institutional email systems and unhardened mail servers, electronic communications suffer from four foundational security vulnerabilities:

- **Opportunistic Cleartext Fallbacks & STARTTLS Stripping (RFC 3207):** Traditional Mail Transfer Agents (MTAs) negotiate encryption opportunistically via the `STARTTLS` command on Port 25. If an active network attacker performs a STARTTLS-stripping attack, the communication channel silently falls back to unencrypted cleartext ASCII transmission, exposing login credentials and confidential student records.
- **Email Spoofing, Header Discrepancy & Sender Impersonation:** Legacy SMTP inherently decouples the transport envelope sender (`MAIL FROM:`) from the presentation header (`From:`). Because traditional MTAs accept incoming connections without cryptographically validating domain ownership, external malicious actors can inject forged emails claiming to originate from college administrators (e.g., `director@jims.edu`, `placement@jims.edu`).
- **Surveillance via Hidden Tracking Beacons (Web Bugs / Spy Pixels):** Inbound marketing and spear-phishing emails frequently embed invisible 1×1 or 0×0 transparent GIF/PNG image tags. When a recipient opens the message, the client automatically executes an outbound HTTP GET request, covertly leaking the user's public IP address, physical location, and reading behavior.
- **Mutable, Non-Verifiable Server Logs:** Standard mail servers write events to flat ASCII text files (`/var/log/mail.log`). These files lack cryptographic hashing or forward secrecy; rogue users or compromised root accounts can erase, truncate, or tamper with logs without detection.
- **Prohibitive Recurring Licensing Costs:** Commercial cloud services charge $6.00 to $12.00 per mailbox monthly. For an institution with 2,000 accounts, this creates an annual financial drain of ₹1.2 Crore to ₹2.4 Crore.

3. Description of the Proposed System

The proposed system, **SecureMail**, is an integrated, self-hosted, cloud-capable web email platform engineered to eliminate the architectural flaws of traditional email transport. Engineered specifically for institutional and academic networks, SecureMail establishes a sovereign, zero-trust communication infrastructure.

Core Architectural Features:

4. Description and Identification of Functional Modules

The system is organized into five tightly coupled functional modules:

1. **Mail Ingestion & Transport Security Engine:** Manages SMTP socket negotiations, enforcing mandatory TLS 1.3 with AES-256-GCM cipher armor and terminating unencrypted connections at the socket layer.
2. **Anti-Spoofing & Identity Verification Gateway:** Validates incoming DNS SPF TXT records, verifies 2048-bit OpenDKIM cryptographic signatures, and strictly enforces DMARC `p=reject` alignment.
3. **In-Memory Privacy Sanitizer:** High-performance regular expression stream parser that detects and neutralizes zero-dimension (`0x0`, `1x1`) tracking beacons in RAM within microseconds.
4. **Cryptographic Audit Ledger Module:** Computes deterministic SHA-256 block hashes linking sequential security events into an immutable, chained audit ledger for forensic non-repudiation.
5. **Webmail Client & Mailbox Management Portal:** Delivers an intuitive three-pane webmail interface and cross-platform mobile client with real-time security badges.

Identification of Users & Stakeholders

- **Campus Students:** Access secure webmail and mobile client, compose and receive encrypted messages, view security badges, and exchange academic assignments safely.
- **Faculty Members & Mentors:** Broadcast verified departmental communications and coordinate academic reviews without spoofing risk.
- **Administrative Officers:** Dispatch sensitive institutional notices, fee receipts, and placement offer letters.
- **System Administrators:** Monitor daemon health, manage mailbox quotas, and mathematically verify audit ledger integrity.

Security Defense-in-Depth Pipeline Flow

Every transaction proceeds through five sequential security gates:

```
[Inbound Socket] → Gate 1: Mandatory TLS 1.3 Handshake (AES-256-GCM)
                 → Gate 2: SPF + 2048-bit OpenDKIM + DMARC p=reject
                 → Gate 3: In-Memory Regex Tracking Pixel Purging
                 → Gate 4: Dovecot Maildir Local Mailbox Storage
                 → Gate 5: SHA-256 Merkle Chained Audit Ledger Logging
```

5. Tools / Platforms

5.1 Hardware Specifications

Hardware Resource	Minimum Specification	Recommended Specification
Processor (CPU)	64-bit Dual-Core @ 2.0 GHz	64-bit Quad-Core @ 2.4 GHz+ with AES-NI Hardware Acceleration
System Memory (RAM)	4 GB ECC DDR4	8 GB – 16 GB ECC DDR4 (Dedicated daemon buffer allocation)
Permanent Storage	40 GB SSD Storage	120 GB – 250 GB NVMe SSD (Exceeding 50,000 IOPS for Maildir)
Network Interface	100 Mbps Ethernet Controller	1 Gbps Full-Duplex NIC with Dedicated Static Public IPv4

5.2 Software Specifications

Subsystem Layer	Technology Selected	Architectural Purpose
Operating System	Ubuntu 22.04 LTS (Jammy)	Linux Kernel 6.1, high-concurrency epoll socket support, AppArmor
Mail Transfer Agent (MTA)	Postfix 3.8	Least-privilege routing daemon hardened with mandatory TLS 1.3
Mail Delivery Agent (MDA)	Dovecot 2.3	RFC 3501 IMAP compliance, Maildir single-file storage, LMTP delivery
Cryptographic Identity	OpenDKIM 2.11	Reference RFC 6376 implementation, 2048-bit RSA key signing/verifying
Backend Application	Python 3.11 / Django 5.0	High-performance runtime, built-in CSRF/XSS protection, REST API
Database Engine	PostgreSQL 15 / SQLite3	ACID compliance, Write-Ahead Logging (WAL), B-Tree indexing
Frontend Technologies	HTML5, CSS3, JavaScript (ES6+)	Standards-based responsive UI without commercial tracker bloat
Mobile Application	Flutter 3.19 (Dart 3.3)	Native ARM64 compilation for Android/iOS with encrypted caching

6. Methodology

6.1 SDLC Model to be Used

The project adopts an **Iterative Waterfall / Agile Hybrid Model**. This methodology divides the project life cycle into disciplined, sequential phases (System Analysis, Protocol Hardening, Subsystem Implementation, Security Verification, and Final Documentation) while enabling iterative micro-cycles for testing individual cryptographic filters and user interface components.

6.2 Justification for the Selection of Model

Developing an enterprise email system requires rigorous network-level protocol compliance (RFC 8446 for TLS 1.3, RFC 6376 for DKIM, and RFC 7489 for DMARC). The iterative hybrid approach allows transport sockets and Milter filters to be rigorously tested for socket stability and RFC adherence before proceeding to higher-level web interface construction, ensuring zero architectural regressions.

6.3 Implementation Timeline & Roadmap

The implementation schedule spans the 12-week MCA Semester-III calendar: Weeks 1–2 (Requirements & Threat Modeling), Weeks 3–4 (MTA Socket Hardening & TLS 1.3 Binding), Weeks 5–6 (DNS SPF/DKIM/DMARC Gateway), Weeks 7–8 (In-Memory Privacy Filter & Merkle Ledger), Weeks 9–10 (Webmail Portal & Flutter App Integration), Weeks 11–12 (Test Matrix Execution & Final Review).

7. Future Scope of the Project

- **Automated Antivirus Attachment Inspection:** Integrating an in-line ClamAV scanning daemon to automatically quarantine malicious attachments and macro-infected documents.
- **Client-Side WebAssembly OpenPGP Encryption:** Implementing client-side OpenPGP key encryption in the browser for end-to-end zero-knowledge confidential communication.
- **Multi-Campus Domain Routing:** Expanding the administrative console to manage multi-tenant domain trees (`@jimsindia.org`, `@bca.jimsindia.org`) from a single control panel.
- **Machine Learning Phishing Classifier:** Embedding localized NLP heuristic classifiers to identify targeted spear-phishing and social engineering attacks.

8. Key References

1. E. Rescorla, "The Transport Layer Security (TLS) Protocol Version 1.3," IETF RFC 8446, Aug. 2018.
2. D. Crocker et al., "DomainKeys Identified Mail (DKIM) Signatures," IETF RFC 6376, Sep. 2011.
3. M. Kucherawy and E. Zwicky, "Domain-based Message Authentication, Reporting, and Conformance (DMARC)," IETF RFC 7489, Mar. 2015.

4. R. C. Merkle, "A digital signature based on a conventional encryption function," in *Advances in Cryptology — CRYPTO '87*, Springer-Verlag, 1988, pp. 369–378.
5. R. S. Pressman and B. R. Maxim, *Software Engineering: A Practitioner's Approach*, 9th ed. McGraw-Hill Education, 2020.